

Data Structure(CSL202)

By-
Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)



Queue

Queue

- A Queue is a linear data structure where:
  - Insertion happens at the rear (end)
  - Deletion happens at the front (start)
- It follows the First In, First Out (FIFO) principle:





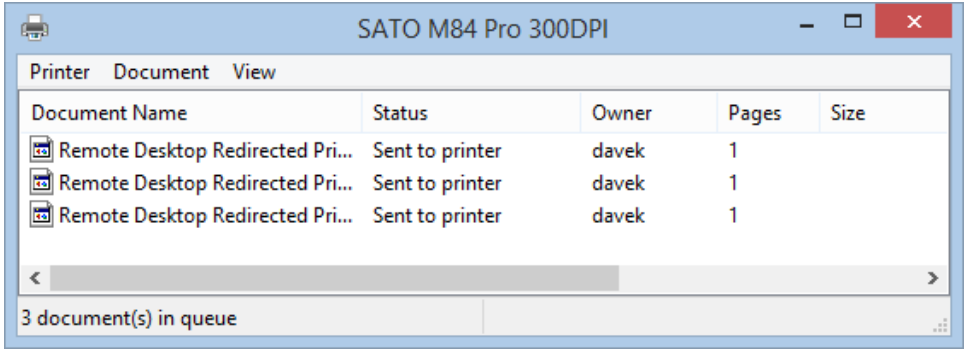


Applications of Queues

Spooling

- Print jobs sent to a printer are handled in the order they were requested.





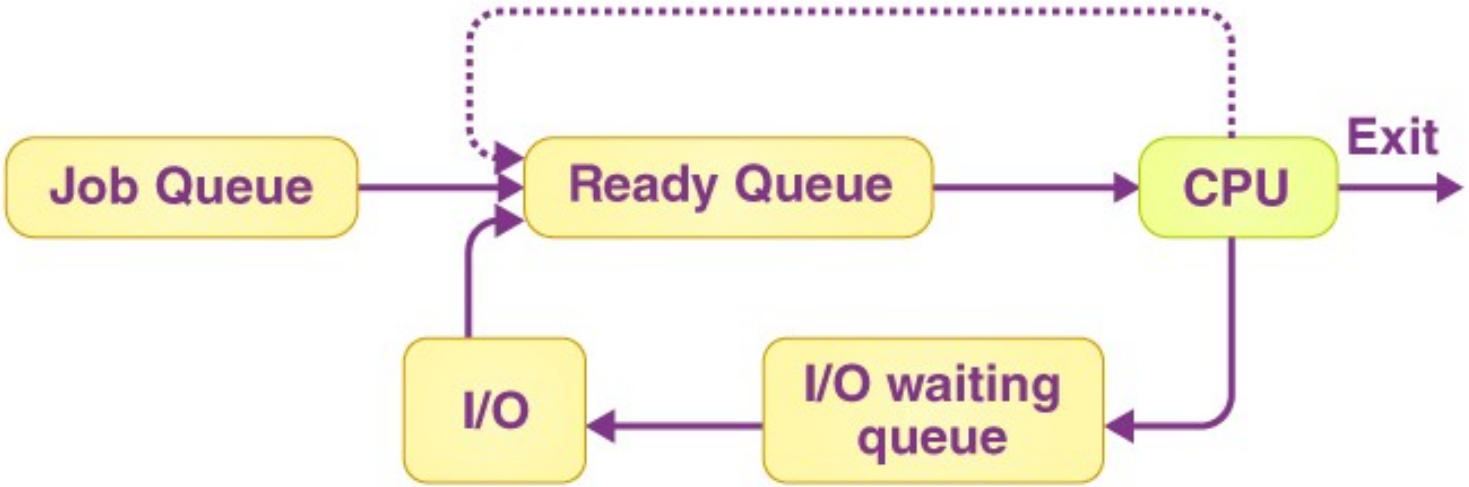
Ticketing Systems

- IRCTC ticket booking uses a queue to process booking and waiting list.



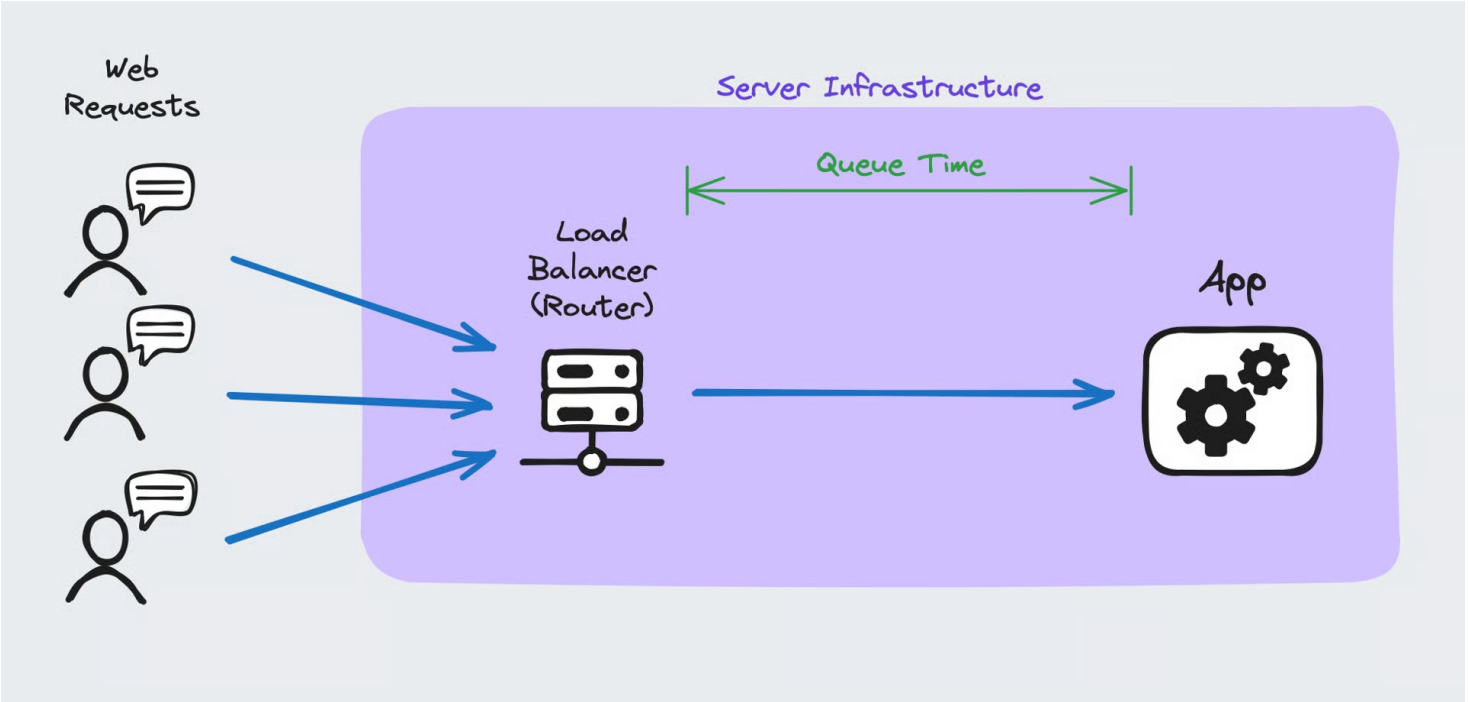
Task Scheduling in Operating Systems

- Processes wait in a queue for CPU allocation.



Web Server Request Queues

- HTTP requests processed in arrival order.





Representation of Queue

Array Representation of Queues

Enqueue:

```
1. [Check for Overflow]
   If (FRONT = 1 AND REAR = N) OR
   (FRONT = REAR + 1), then:
      Write: "OVERFLOW"
      Return
2. [Find new REAR position]
   If FRONT = NULL, then:
      Set FRONT := 1
   Set REAR := 1
   Else if REAR = N, then:
      Set REAR := 1
   Else:
      Set REAR := REAR + 1
3. [Insert ITEM]
   Set QUEUE[REAR] := ITEM
4. Return
```

Array Representation of Queues

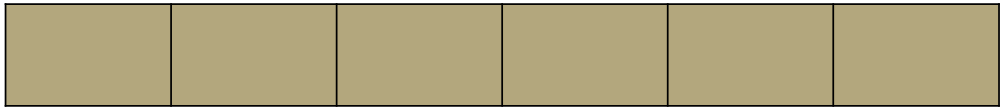
Dequeue:

1. [Check for Underflow]
   If FRONT = NULL, then:
      Write: "UNDERFLOW"
      Return
2. [Access front element]
   Set ITEM := QUEUE[FRONT]
3. [Update FRONT]
   If FRONT = REAR, then:
      Set FRONT := NULL
      Set REAR := NULL
   Else if FRONT = N, then:
      Set FRONT := 1
   Else:
      Set FRONT := FRONT + 1
4. Return

Array Representation of Queues

FRONT = 0

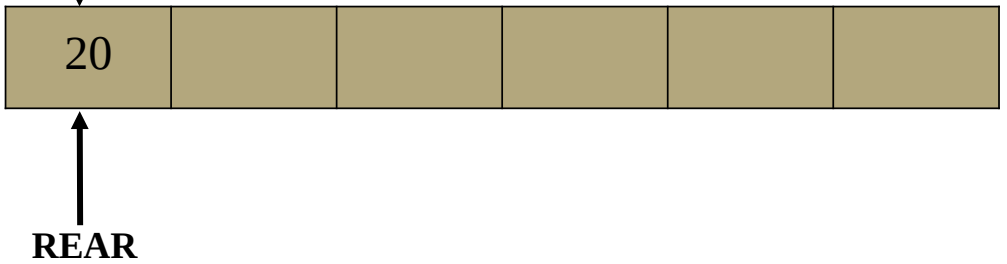
Initially



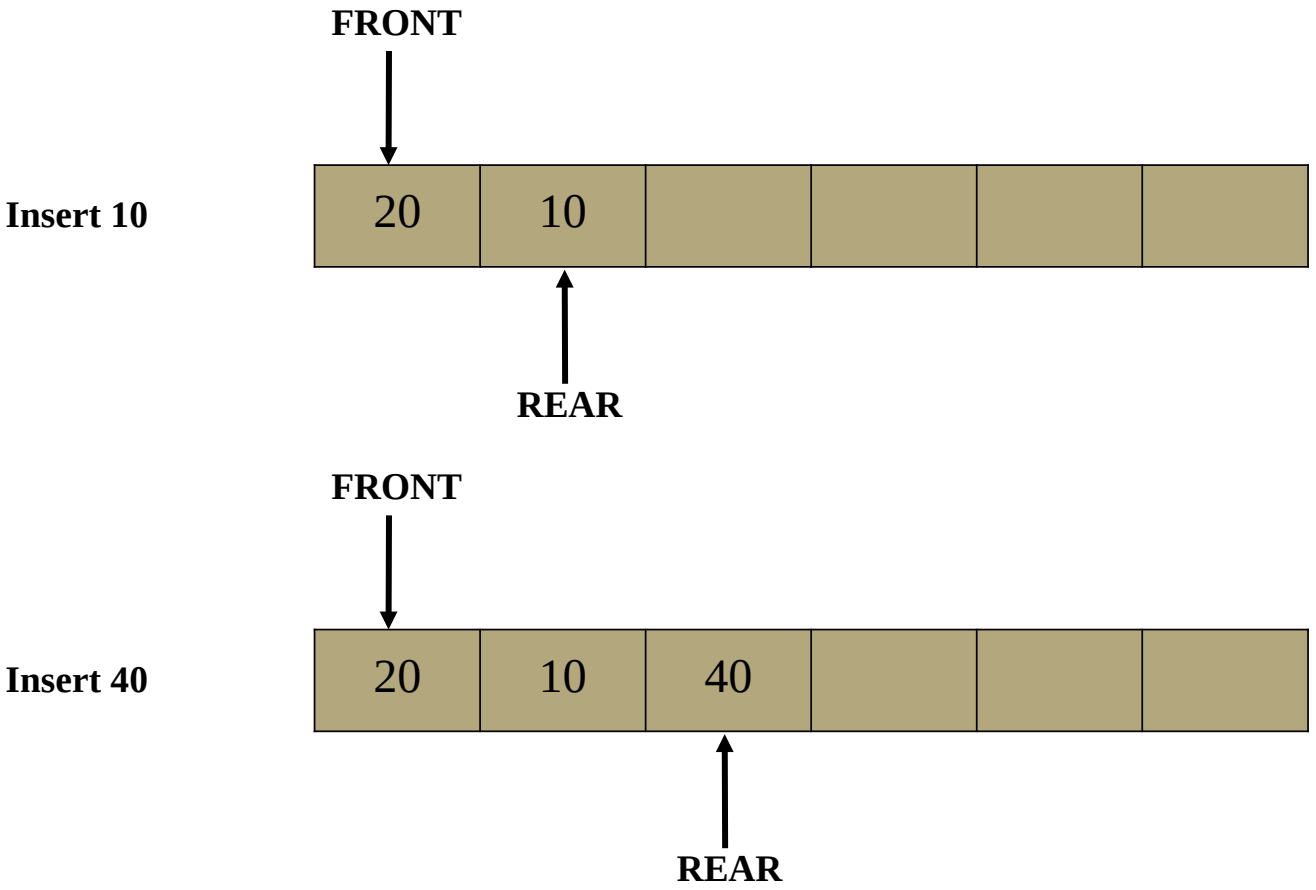
REAR = 0

FRONT

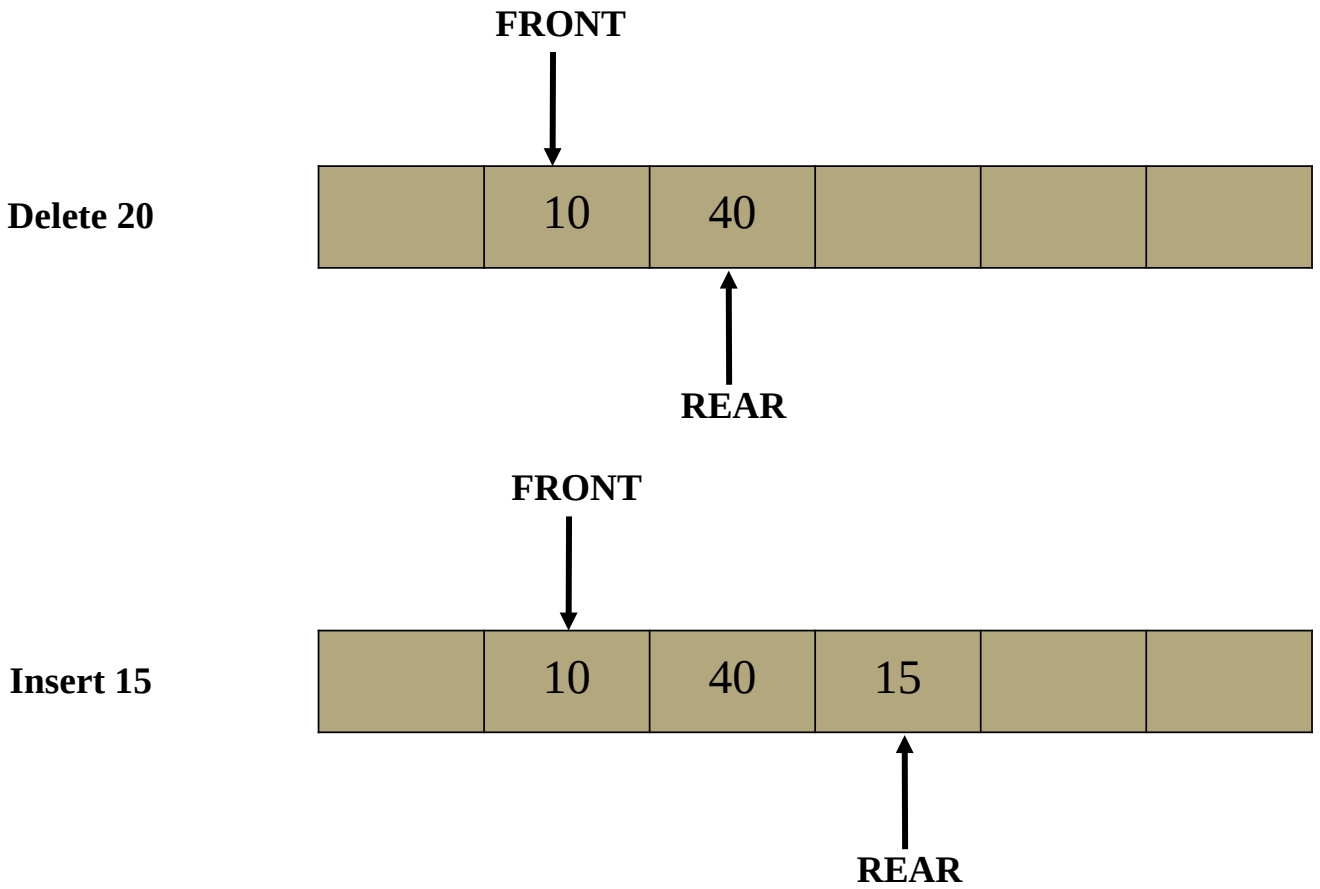
Insert 20



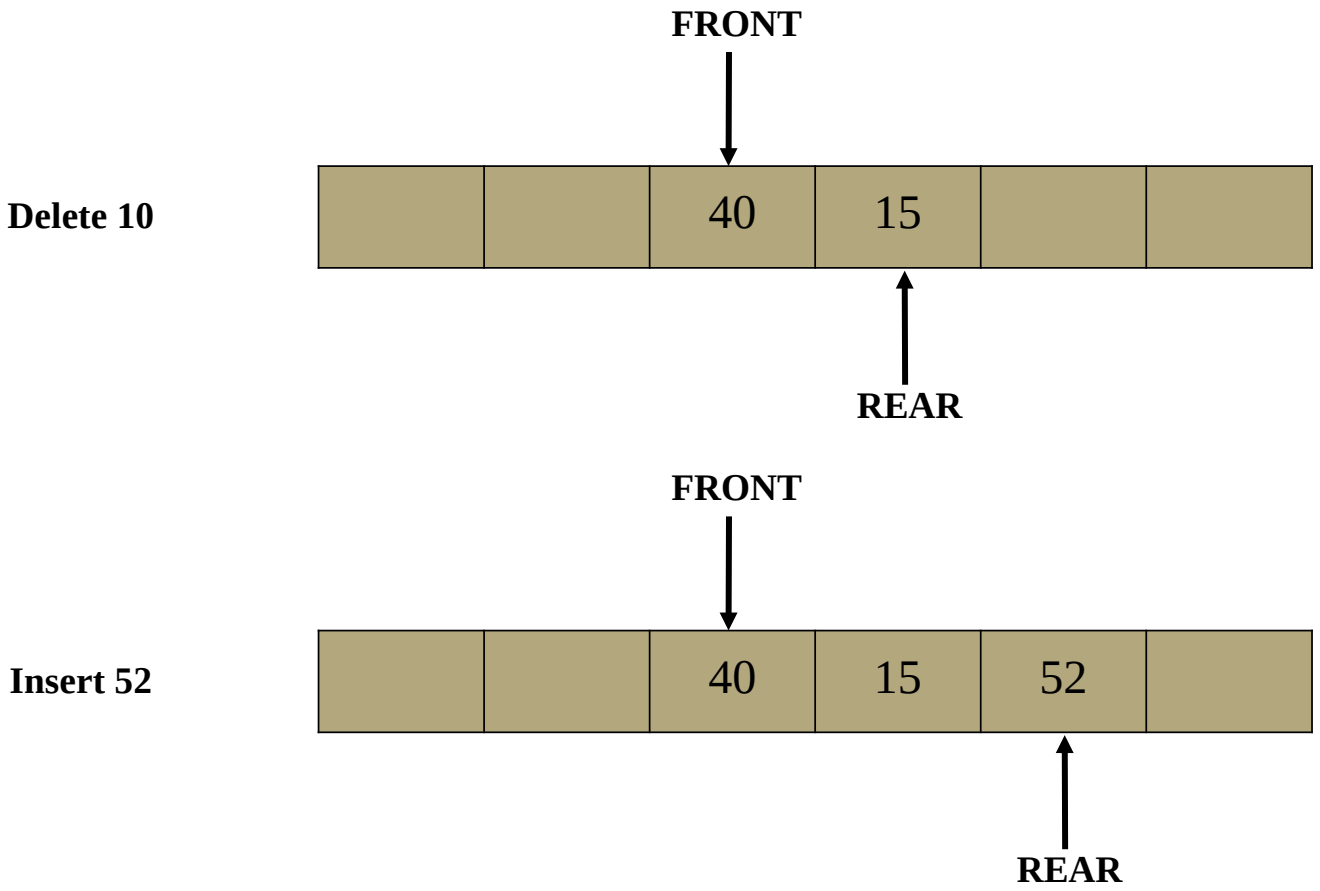
Array Representation of Queues



Array Representation of Queues



Array Representation of Queues



Array Representation of Queues Complexity

Enqueue:

- Insertion in a queue always takes O(1) time, regardless of the case, since it involves a fixed number of pointer updates and array assignments.
  - **Worst-** Insertion when REAR wraps around to the front.O(1)
  - **Average:** Typical insertion, involves REAR adjustment.O(1)
  - **Best:** Direct insertion without wrapping around. O(1)

Array Representation of Queues Complexity

Dequeue:

- Deletion from a queue always takes O(1) time, regardless of the case, since it involves a fixed number of pointer updates and array assignments.
  - **Worst-** O(1)
  - **Average:** O(1)
  - **Best:** O(1)

Linked List Representation of Queues

Enqueue:

1. **Check for available space**

   If AVAIL = NULL, print OVERFLOW and exit.

2. **Allocate a new node**

   Set NEW ← AVAIL

   Update AVAIL ← LINK[AVAIL]

3. **Store the item**

   INFO[NEW] ← ITEM

   LINK[NEW] ← NULL

4. **Insert node at rear**

   If the queue is empty (FRONT = NULL):

   a. Set FRONT ← NEW

   b. REAR ← NEW

   Else:

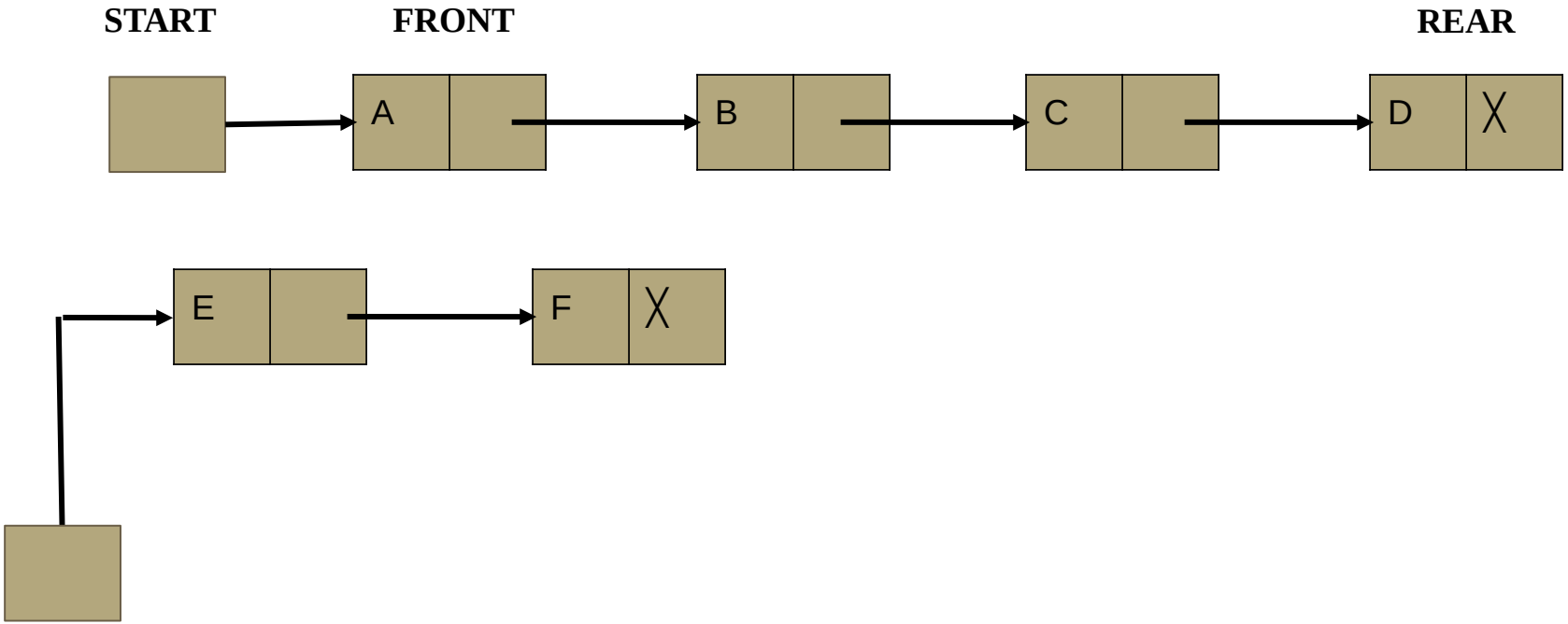
   c. LINK[REAR] ← NEW

   d. REAR ← NEW

5. **End procedure**

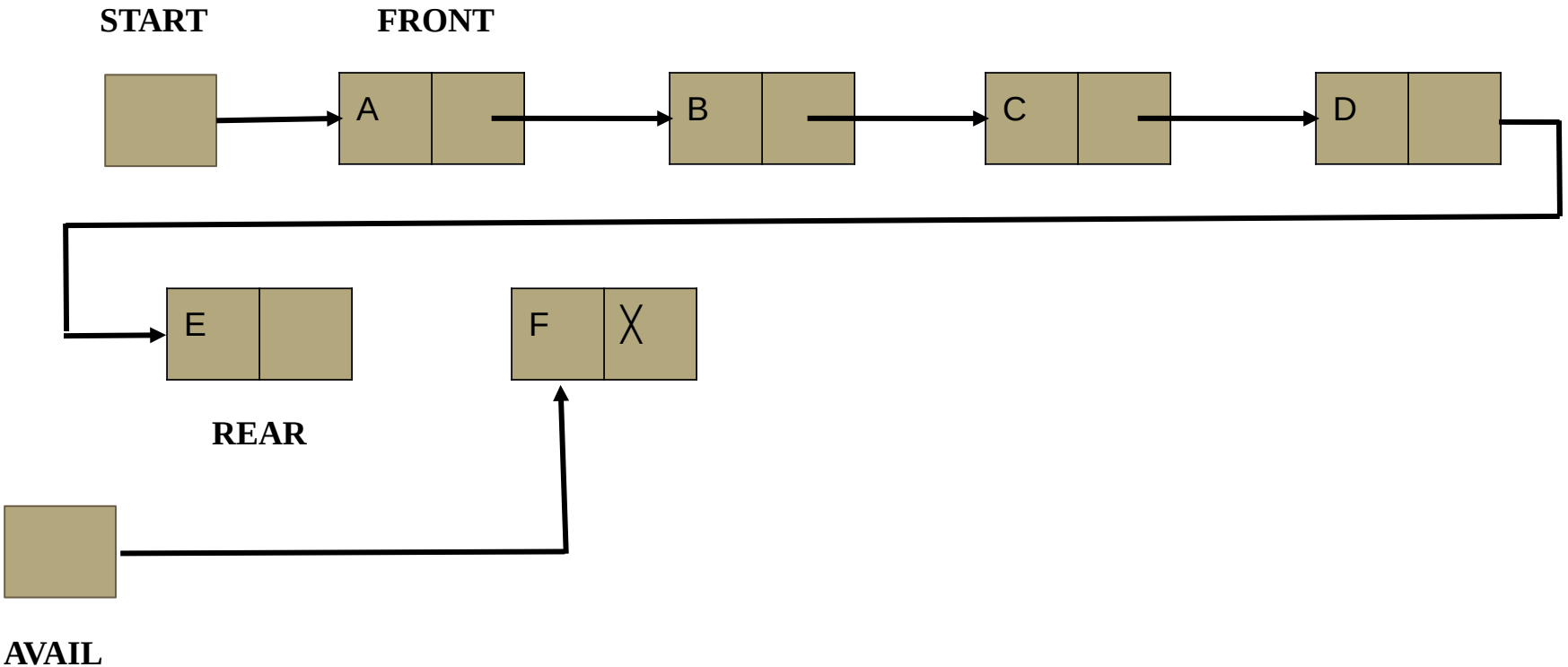
Linked List Representation of Queues

Enqueue:



Linked List Representation of Queues

Enqueue:



Linked List Representation of Queues

Linked List Representation of Queues Complexity

Enqueue:

- All operations involve pointer adjustments only — no traversal needed.
  - **Worst-** O(1)
  - **Average:** O(1)
  - **Best:** O(1)

Linked List Representation of Queues

Dequeue:

1. **Check if queue is empty**

   If FRONT = NULL, print UNDERFLOW and exit.

2. **Store the front node temporarily**

   Set TEMP ← FRONT.

3. **Copy the front item**

   Set ITEM ← INFO[TEMP].

4. **Advance FRONT pointer**

   Set FRONT ← LINK[TEMP].

5. **Return deleted node to AVAIL list**

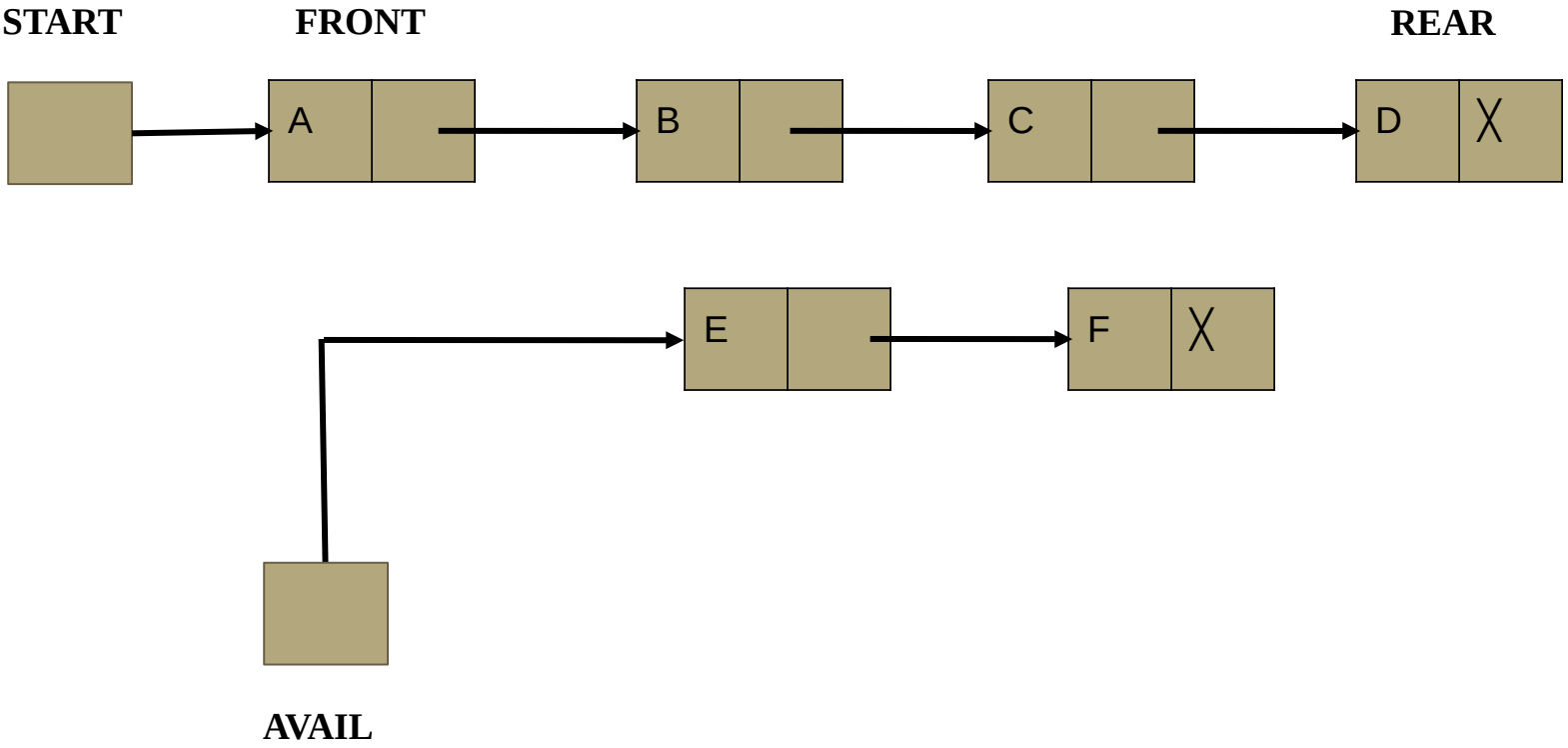
   Set LINK[TEMP] ← AVAIL.

   Set AVAIL ← TEMP.

6. **Exit procedure**

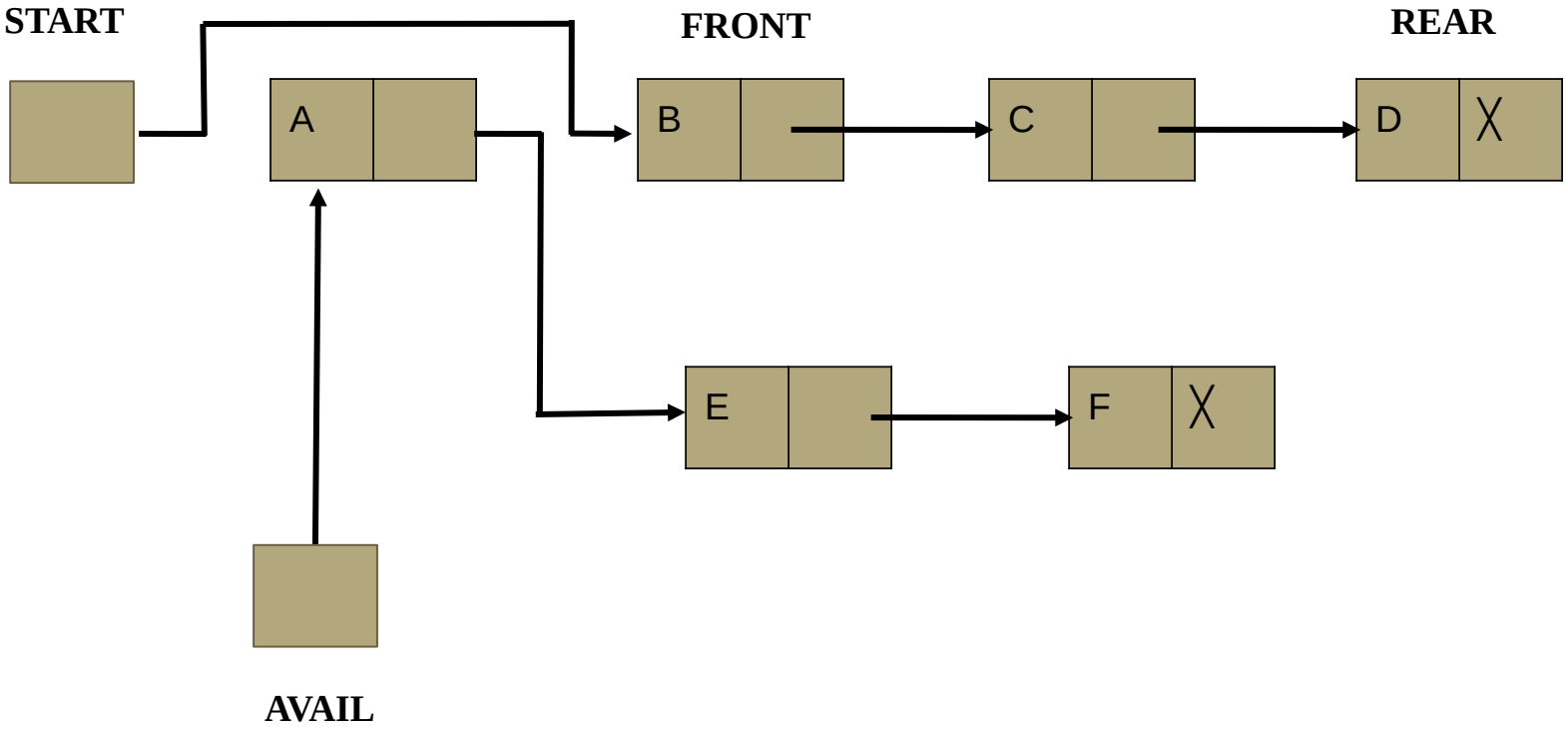
Linked List Representation of Queues

Dequeue:



Linked List Representation of Queues

Dequeue:



Linked List Representation of Queues Complexity

Dequeue:

- Deletion affects only the front node and involves basic pointer changes without traversal.
  - **Worst-** O(1)
  - **Average:** O(1)
  - **Best:** O(1)